

A
Project Report
On
**“Design a reliable message delivery system over UDP using
sequence numbers and acknowledgments.”**



Department Of Computer Science & Engineering
National Institute of Technology, Patna

Submitted By:

Name	Roll No.
Suvir Shah	2406018
Harsh Kapadiya	2406046
Surya Kumar Yadav	2406072

Course Code: CS44113

Course Title: Computer Networks

Branch: Computer Science and Engineering

TABLE OF CONTENTS

- 1. Aim of the Experiment**
- 2. Introduction**
 - 2.1 Background
 - 2.2 Problem Statement
 - 2.3 Objective
 - 2.4 Scope and Limitations
- 3. Theoretical Background**
 - 3.1 The UDP Protocol
 - 3.2 The Need for Reliable Data Transfer
 - 3.3 Stop-and-Wait Protocol
 - 3.4 Sequence Numbers and Acknowledgments
- 4. Requirements**
 - 4.1 Hardware Requirements
 - 4.2 Software Requirements
 - 4.3 Functional Requirements
 - 4.4 Non-Functional Requirements
- 5. Data Flow Diagram**
 - 5.1 System Overview
 - 5.2 Sender Flowchart
 - 5.3 Receiver Flowchart
- 6. Implementation (Code)**
 - 6.1 Receiver Program
 - 6.2 Sender Program
- 7. Output**
- 8. Observations**
- 9. Conclusion**
- 10. Learning Outcomes**
- 11. References**

Aim of the experiment:

Design a reliable message delivery system over UDP using sequence numbers and acknowledgments.

1. Introduction:

1.1 Background

In modern computer networks, data communication relies on transport layer protocols to ensure correct delivery of information. The two primary transport protocols are TCP and UDP. TCP provides reliable, ordered, and error-checked delivery through mechanisms such as connection establishment, flow control, and congestion control. However, these features introduce additional overhead.

UDP, in contrast, is a lightweight, connectionless protocol that does not guarantee delivery, ordering, or data integrity. Due to its low overhead and faster transmission, UDP is widely used in time-sensitive applications such as online gaming, video streaming, VoIP, and DNS services.

1.2 Problem Statement

Despite its efficiency, UDP has several limitations that make it unsuitable for reliable communication. These include:

- Packet Loss: Data packets may be dropped without notification
- Out-of-Order Delivery: Packets may arrive in a different order
- Data Corruption: Errors in transmission may go undetected

These limitations require the implementation of reliability mechanisms at the application layer.

1.3 Objective

The objective of this project is to design and implement a reliable message delivery system over UDP using sequence numbers and acknowledgments. The system uses the Stop-and-Wait ARQ protocol to ensure that each packet is delivered correctly and in order. If an acknowledgment is not received within a specified timeout, the packet is retransmitted.

The system is implemented in Python using UDP sockets and includes mechanisms such as sequence numbering, acknowledgments, retransmission, and checksum-based error detection.

1.4 Scope and Limitations

This implementation has the following scope:

- Single sender and single receiver communication
- Use of Stop-and-Wait protocol (one packet at a time)

Limitations include lower efficiency compared to sliding window protocols and limited error detection capability of the checksum.

2. Theoretical Background

2.1 The UDP Protocol

The User Datagram Protocol (UDP) is a transport layer protocol that provides a connectionless communication model. Unlike TCP, UDP does not establish a connection before transmitting data and does not guarantee reliable delivery.

The UDP header is minimal and consists of four fields:

- Source Port
- Destination Port
- Length
- Checksum

Due to its simplicity, UDP has low overhead and high speed. However, it does not provide mechanisms for packet acknowledgment, retransmission, or ordering. As a result, packets may be lost, duplicated, or received out of order.

UDP is widely used in applications where low latency is critical, such as online gaming, video streaming, Voice over IP (VoIP), and Domain Name System (DNS) services.

2.2 The Need for Reliable Data Transfer

Reliable data transfer ensures that data is delivered:

- Without loss
- In the correct order
- Without duplication

In many applications, such as file transfer or messaging systems, reliability is essential. However, UDP does not provide these guarantees by default. Therefore, reliability must be implemented at the application layer when using UDP.

This creates a need for mechanisms that can detect lost packets, maintain correct order, and ensure successful delivery.

Automatic Repeat reQuest (ARQ) is a class of protocols used to achieve reliable data transmission. ARQ protocols rely on acknowledgments (ACKs) and retransmissions.

If an acknowledgment is not received within a certain time, the sender retransmits the packet.

2.3 Stop-and-Wait Protocol

The Stop-and-Wait protocol is the simplest form of ARQ. In this protocol, the sender transmits one packet at a time and waits for its acknowledgment before sending the next packet.

The working is as follows:

1. Sender sends a packet
2. Sender waits for acknowledgment
3. If ACK is received, next packet is sent
4. If ACK is not received within a timeout, the packet is retransmitted

This ensures reliable delivery but is inefficient because only one packet is in transit at any time.

2.4 Sequence Numbers and Acknowledgments

Sequence numbers are used to uniquely identify packets. They help the receiver:

- Detect duplicate packets
- Maintain correct order of data

Acknowledgments (ACKs) are sent by the receiver to confirm successful receipt of a packet. The sender uses these ACKs to decide whether to proceed or retransmit.

Together, sequence numbers and acknowledgments form the core mechanism for achieving reliable communication over unreliable networks.

3. Requirements

3.1 Hardware Requirements

- A computer system with at least:
 1. Processor: Intel i3 or higher
 2. RAM: 4 GB minimum
 3. Storage: 500 MB free space
- Network interface (for UDP communication)

3.2 Software Requirements

- Operating System: Windows / Linux / macOS
- Programming Language: Python 3.x
- Python Libraries:
 - socket (for network communication)
 - time (for timeout handling)

3.3 Functional Requirements

The system should:

- Send messages from sender to receiver using UDP
- Attach sequence numbers to each message
- Wait for acknowledgment before sending the next message
- Retransmit messages if acknowledgment is not received within timeout
- Ensure messages are delivered in correct order
- Detect and handle duplicate or out-of-order packets
- Allow clean termination using a DONE signal

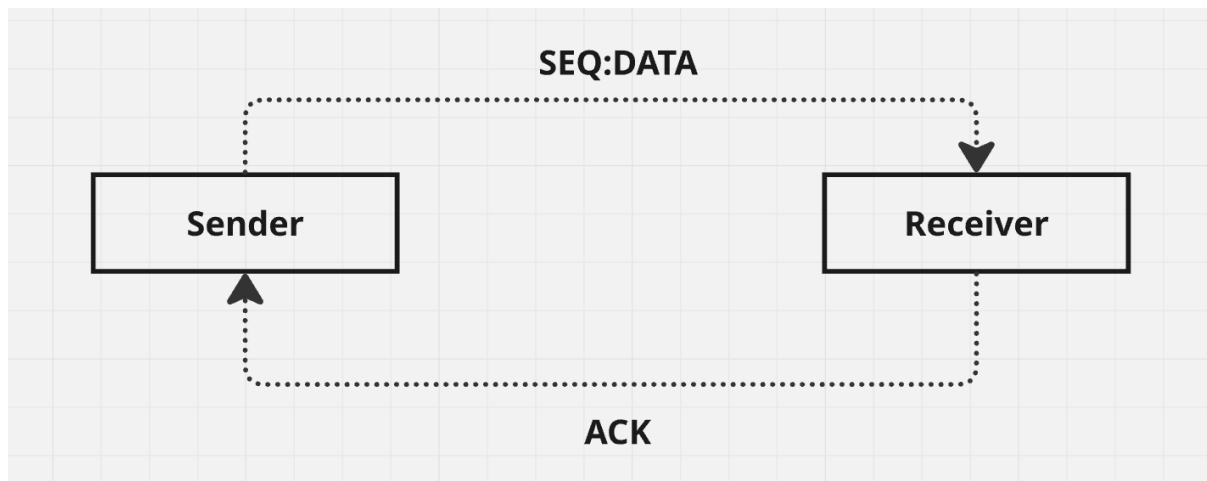
3.4 Non-Functional Requirements

- **Reliability:** System should ensure correct delivery of messages
- **Efficiency:** Should minimize unnecessary retransmissions
- **Scalability:** Designed for small-scale communication (single sender–receiver)
- **Usability:** Simple execution using command line
- **Performance:** Low latency communication using UDP
- **Maintainability:** Code should be readable and modular

4. DATA FLOW DIAGRAM

The system consists of two components:

- Sender: Responsible for transmitting messages
- Receiver: Responsible for receiving messages and sending acknowledgments



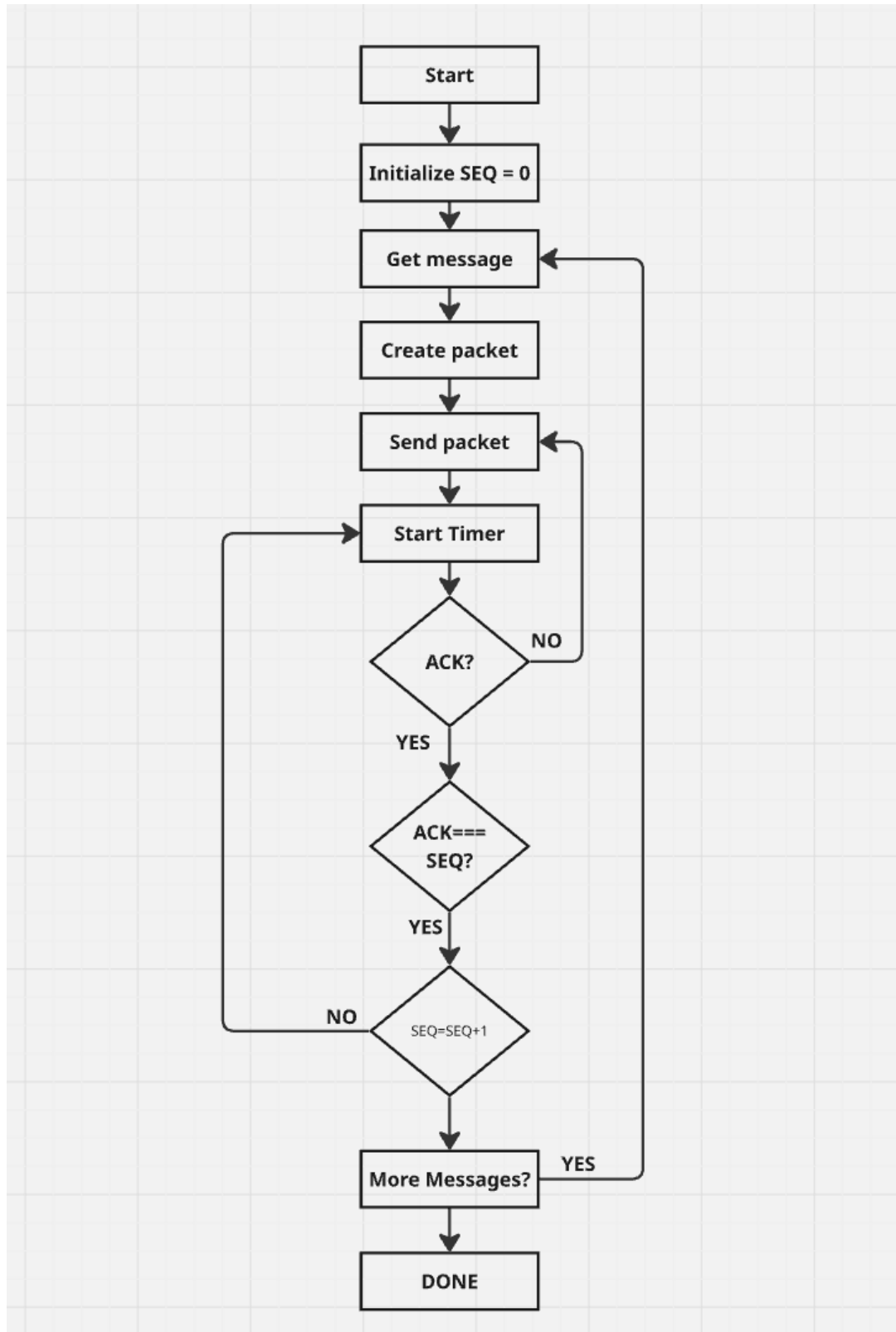
Sender Flowchart

The sender follows these steps:

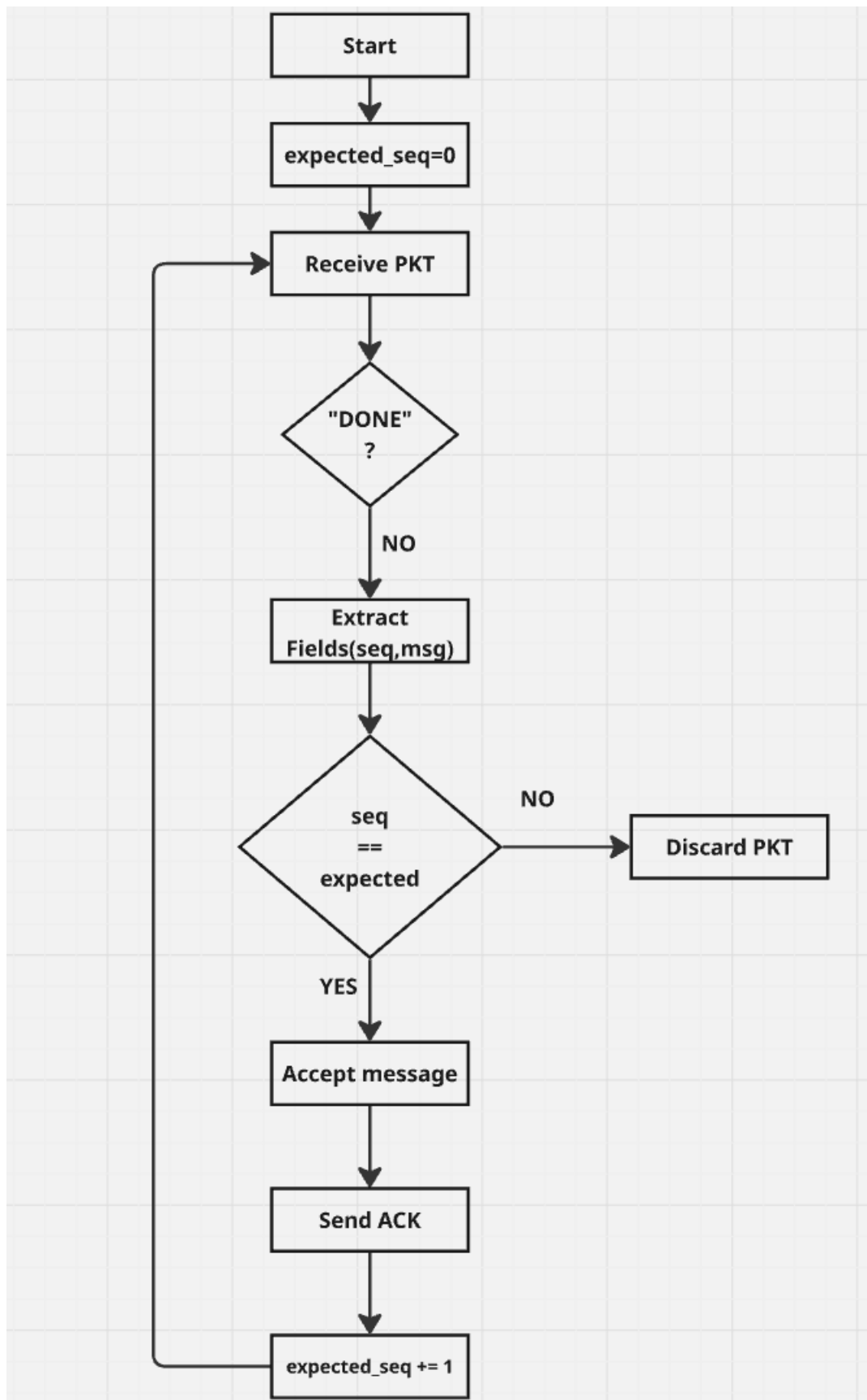
1. Assign a sequence number to the message
2. Create a packet by combining sequence number and message
3. Send the packet to the receiver

4. Start a timer and wait for acknowledgment
5. If ACK is received and matches the sequence number, proceed to the next message
6. If timeout occurs, retransmit the same packet

This process continues until all messages are successfully delivered.



Receiver Flowchart



The receiver performs the following operations:

1. Wait for incoming packets
2. Check if the received message is a DONE signal
3. Extract sequence number and message from the packet
4. Compare sequence number with expected sequence
5. If correct:
 - a. Accept the message
 - b. Send acknowledgment
 - c. Update expected sequence number
6. If incorrect:
 - a. Discard the packet
 - b. Resend last acknowledgment

5. CODE

receiver.py

```
import socket
HOST = '127.0.0.1'
PORT = 5005
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((HOST, PORT))
print("Receiver is running...")
expected_seq = 0
while True:
    data, addr = sock.recvfrom(1024)
    if data == b"DONE":
        print("Transmission complete. Exiting...")
        break
    message = data.decode()
    try:
        seq, msg = message.split(":", 1)
        seq = int(seq)
    except:
        continue
    print(f"Received -> SEQ: {seq}, MSG: {msg}")
    if seq == expected_seq:
        print(f"Accepted: {msg}")
        sock.sendto(str(seq).encode(), addr)
        expected_seq += 1
    else:
        print("Duplicate or out-of-order packet")
        ack = max(0, expected_seq - 1) # avoid -1
        sock.sendto(str(ack).encode(), addr)
sock.close()
print("Receiver closed.")
```

sender.py

```
import socket
import time
HOST = '127.0.0.1'
```

```

PORT = 5005
TIMEOUT = 2
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.settimeout(TIMEOUT)
messages = ["This", "is", "Group", "17's", "Project"]
seq = 0
for msg in messages:
    while True:
        packet = f"{seq}:{msg}"
        print(f"Sending -> SEQ: {seq}, MSG: {msg}")
        sock.sendto(packet.encode(), (HOST, PORT))
        try:
            ack, _ = sock.recvfrom(1024)
            ack = int(ack.decode())
            print(f"Received ACK: {ack}")
            if ack == seq:
                break
        except socket.timeout:
            print("Timeout! Retransmitting...")
        except ConnectionResetError:
            print("Receiver not available! Retrying...")
            time.sleep(1)
    seq += 1
    time.sleep(1)
print("Sending DONE signal...")
sock.sendto(b"DONE", (HOST, PORT))
sock.close()
print("Sender finished.")

```

6. OUTPUT

Sender sending message

```

PS C:\Users\surya\OneDrive\Desktop\2406072\COMP_NETWORK\
Project> python sender.py
Sending -> SEQ: 0, MSG: This
Received ACK: 0
Sending -> SEQ: 1, MSG: is
Received ACK: 1
Sending -> SEQ: 2, MSG: Group
Received ACK: 2
Sending -> SEQ: 3, MSG: 17's
Received ACK: 3
Sending -> SEQ: 4, MSG: Project
Received ACK: 4
Sending DONE signal...
Sender finished.

```

Receiver side:

```
PS C:\Users\surya\OneDrive\Desktop\2406072\COMP2023\Project> python receiver.py
Receiver is running...
Received -> SEQ: 0, MSG: This
Accepted: This
Received -> SEQ: 1, MSG: is
Accepted: is
Received -> SEQ: 2, MSG: Group
Accepted: Group
Received -> SEQ: 3, MSG: 17's
Accepted: 17's
Received -> SEQ: 4, MSG: Project
Accepted: Project
Transmission complete. Exiting...
Receiver closed.
```

When ACK Lost:

```
PS C:\Users\surya\OneDrive\Desktop\2406072\COMP2023\Project> python receiver.py
Receiver is running...
Received -> SEQ: 0, MSG: This
Accepted: This
Received -> SEQ: 1, MSG: is
Accepted: is
Received -> SEQ: 2, MSG: Group
Accepted: Group
Received -> SEQ: 3, MSG: 17's
Accepted: 17's
Received -> SEQ: 4, MSG: Project
Accepted: Project
ACK lost (simulated)
Received -> SEQ: 4, MSG: Project
Duplicate or out-of-order packet
Transmission complete. Exiting...
Receiver closed.
```

```
PS C:\Users\surya\OneDrive\Desktop\2406072\COMP_NETWORK
Project> python sender.py
Sending -> SEQ: 0, MSG: This
Received ACK: 0
Sending -> SEQ: 1, MSG: is
Received ACK: 1
Sending -> SEQ: 2, MSG: Group
Received ACK: 2
Sending -> SEQ: 3, MSG: 17's
Received ACK: 3
Sending -> SEQ: 4, MSG: Project
Timeout! Retransmitting...
Sending -> SEQ: 4, MSG: Project
Received ACK: 4
Sending DONE signal...
Sender finished.
```

To evaluate the reliability of the system, acknowledgment loss was simulated at the receiver side. A random probability was introduced to intentionally drop ACK messages instead of sending them back to the sender.

7. OBSERVATIONS

- In normal conditions, all messages are delivered correctly and in order with proper acknowledgments.
- When ACK loss is simulated, the sender retransmits the packet after timeout, ensuring reliable delivery.
- Duplicate packets may occur due to retransmission, but the receiver correctly discards them using sequence numbers.
- The timeout mechanism ensures that lost packets or acknowledgments do not affect communication.
- Messages are always received in the correct sequence due to the Stop-and-Wait protocol.
- The DONE signal allows the receiver to terminate execution gracefully after all messages are received.

8. CONCLUSION

In this project, a reliable message delivery system was successfully designed and implemented over UDP using Python sockets.

Since UDP does not provide built-in reliability, the system incorporated the Stop-and-Wait ARQ protocol to ensure correct and ordered delivery of messages.

The implementation used sequence numbers and acknowledgments to track packets and confirm successful delivery.

A timeout-based retransmission mechanism was employed to handle packet or acknowledgment loss. Additionally, a DONE signal was used to allow graceful termination of the receiver process.

Testing of the system, including simulated acknowledgment loss, demonstrated that the sender correctly retransmits packets when acknowledgments are not received, and the receiver properly handles duplicate or out-of-order packets.

This confirms that reliable communication can be achieved over an unreliable protocol like UDP by implementing appropriate control mechanisms at the application layer.

9. LEARNING OUTCOMES

Through this experiment, the following concepts and skills were gained:

- Understanding the difference between TCP and UDP and their trade-offs
- Learning how reliability can be implemented at the application layer over UDP
- Gaining knowledge of the Stop-and-Wait ARQ protocol
- Understanding the role of sequence numbers in maintaining order and detecting duplicates
- Learning how acknowledgments are used to confirm successful delivery
- Implementing timeout and retransmission mechanisms for reliability
- Simulating network issues such as acknowledgment loss and observing system behavior
- Developing practical experience with Python socket programming
- Understanding how theoretical networking concepts are applied in real systems

10. REFERENCES

- PyNet Labs, “UDP vs TCP,” Available at: <https://www.pynetlabs.com/udp-vs-tcp/>
- Real Python, “Python Socket Programming,” Available at: <https://realpython.com/python-sockets/>
- W3Schools, “Python Tutorial,” Available at: <https://www.w3schools.com/python/>